

Percona Operator for PostgreSQL

CR Limitations & Manual DBA Workarounds

20 PostgreSQL Administration Tasks Beyond the Custom Resource

Platform: Percona Operator v2.x (based on Crunchy PGO)

Target OS: AlmaLinux 9 / Kubernetes Cluster

PostgreSQL Versions: 14.x — 17.x

Table of Contents

Part I: Overview — What the CR Supports vs. What It Doesn't

Part II: The 20 CR Limitations — Summary Reference Table

Part III: Detailed Workarounds with Commands

1. Schema Management
2. Granular GRANT / REVOKE Privileges
3. User / Role Deletion
4. Database Deletion / Renaming
5. Extension Installation and Management
6. Table, Index, and DDL Operations
7. VACUUM, ANALYZE, and Maintenance Tasks
8. Query Performance Tuning
9. Logical Replication
10. Row-Level Security (RLS) Policies
11. Tablespace Management (Logical)
12. Foreign Data Wrappers (FDW)
13. Event Triggers and Regular Triggers
14. Scheduled Jobs Inside PostgreSQL (pg_cron)
15. Audit Configuration (pgAudit)
16. Connection-Level Parameter Overrides
17. Large Object Management
18. Database Cloning / Template Databases
19. Statistics and Planner Hints
20. Patroni Direct Interventions

Part IV: Automation Patterns — CronJob, Bastion, and CI/CD

Part V: Common kubectl exec Wrapper Script

Part VI: Quick Reference Cheat Sheet

PART I — OVERVIEW

What the CR Supports vs. What It Doesn't

The Percona Operator for PostgreSQL (v2.x, based on Crunchy PGO) manages PostgreSQL clusters declaratively through a **PerconaPGCluster** Custom Resource. The operator excels at infrastructure-plane tasks — cluster provisioning, replica scaling, HA via Patroni, automated backups via pgBackRest, TLS management, and connection pooling via pgBouncer.

What the CR Does Support

- Cluster provisioning, scaling replicas up/down, resource limits
- User and database creation with basic ALTER ROLE options (CREATEDB, CREATEROLE, etc.)
- Automatic schema creation per user (autoCreateUserSchema)
- Scheduled and on-demand pgBackRest backups (full, differential, incremental) with PITR
- pgBouncer connection pooling configuration
- TLS / cert-manager integration for encrypted connections
- Custom postgresql.conf parameters via the CR's customConfig section
- PostgreSQL major version upgrades (in-place)
- PMM (Percona Monitoring and Management) integration
- Custom extensions via S3-based extension loader
- Patroni HA configuration (switchover enable/disable, replica methods)
- Tablespace PVC volume definitions
- Pod affinity, tolerations, node selectors, sidecar containers
- Graceful cluster shutdown and restart (spec.pause)

What the CR Does Not Support

The CR has no visibility or control over anything below the database level. The entire **data plane** — schemas, object privileges, DDL, DML, extensions (CREATE/ALTER/DROP), maintenance, performance tuning, logical replication, auditing, in-database scheduling, and many lifecycle operations — remains traditional DBA territory requiring direct SQL access to the pods via `kubectl exec`.

The following 20 categories represent the complete set of PostgreSQL DBA operations that fall outside the operator's declarative boundary. Each requires manual SQL execution, scripting, or Kubernetes CronJob automation.

PART II — THE 20 CR LIMITATIONS — SUMMARY REFERENCE

#	Limitation Area	Key Operations Not in CR	Impact
1	Schema Management	CREATE/ALTER/DROP SCHEMA, search_path	High
2	Granular GRANT/REVOKE	Object, column, schema-level privileges	Critical
3	User/Role Deletion	DROP OWNED, DROP ROLE	High
4	Database Deletion/Renaming	DROP DATABASE, ALTER DATABASE RENAME	Medium
5	Extension Management	CREATE/ALTER/DROP EXTENSION	High
6	Table, Index, DDL	All DDL operations, partitioning, views	Critical
7	VACUUM/ANALYZE/Maintenance	VACUUM FULL, REINDEX, CLUSTER	Critical
8	Query Performance Tuning	pg_stat_statements, EXPLAIN, kill queries	Critical
9	Logical Replication	Publications, subscriptions, logical slots	High
10	Row-Level Security	CREATE/ALTER POLICY, ENABLE RLS	Medium
11	Tablespace Mgmt (Logical)	CREATE TABLESPACE, SET TABLESPACE	Medium
12	Foreign Data Wrappers	CREATE SERVER, FOREIGN TABLE, USER MAPPING	Medium
13	Triggers (Event + Regular)	CREATE TRIGGER, EVENT TRIGGER, functions	High
14	pg_cron / In-DB Scheduling	Job creation, scheduling inside PostgreSQL	High
15	Audit Config (pgAudit)	Extension install, per-role audit settings	High
16	Connection-Level Params	ALTER ROLE/DATABASE SET param = value	Medium
17	Large Object Management	lo_import, lo_export, vacuumlo	Low
18	DB Cloning / Templates	CREATE DATABASE ... TEMPLATE ...	Medium
19	Statistics / Planner Hints	ALTER TABLE SET STATISTICS, CREATE STATISTICS	Medium
20	Patroni Direct Actions	patronictl switchover/failover/reinit	High

Critical	Core daily DBA tasks — unavoidable manual work
High	Frequent operational need — scripting recommended
Medium	Periodic tasks — document procedures for team
Low	Rare edge cases — handle ad-hoc

PART III — DETAILED WORKAROUNDS WITH COMMANDS

All commands below assume you have identified the primary pod. Use this preamble in every session:

```

NAMESPACE="pg-production"
CLUSTER="my-cluster"

# Identify the primary pod
PRIMARY=$(kubectl get pod -n "$NAMESPACE" \
-l postgres-operator.crunchydata.com/role=master,\
postgres-operator.crunchydata.com/cluster="$CLUSTER" \
-o jsonpath='{.items[0].metadata.name}')

# Helper function
run_sql() {
  kubectl exec -n "$NAMESPACE" "$PRIMARY" -c database -- \
  psql -U postgres -d "${2:-postgres}" -c "$1"
}

```

Caution: Changes made via `kubectl exec` are NOT tracked by the operator and may be lost during pod recreation or failover. For durable user/database changes, always prefer the CR when possible.

1

Schema Management

The CR auto-creates one schema per user (matching the username) when `autoCreateUserSchema: true`. It cannot create custom schemas, alter them, drop them, or configure `search_path`.

Create Custom Schemas

```

# Create schemas in a specific database
run_sql "CREATE SCHEMA IF NOT EXISTS app;" mydb
run_sql "CREATE SCHEMA IF NOT EXISTS audit;" mydb
run_sql "CREATE SCHEMA IF NOT EXISTS staging;" mydb

# Grant usage to a role
run_sql "GRANT USAGE ON SCHEMA app TO app_user;" mydb
run_sql "GRANT CREATE ON SCHEMA app TO app_user;" mydb

```

Set search_path Per Role or Database

```

# Per-role search_path
run_sql "ALTER ROLE app_user SET search_path = app, public;"

# Per-database search_path
run_sql "ALTER DATABASE mydb SET search_path = app, public;"

```

Drop a Schema

```
# Drop with cascade (DANGEROUS – drops all objects inside)
run_sql "DROP SCHEMA IF EXISTS staging CASCADE;" mydb
```

2 Granular GRANT / REVOKE Privileges

The CR only supports basic ALTER ROLE options like CREATEDB, CREATEROLE, NOSUPERUSER. All object-level, schema-level, column-level grants, and default privileges must be managed via SQL.

Schema-Level and Table-Level Grants

```
# Grant read-only access to all current tables
run_sql "
GRANT USAGE ON SCHEMA app TO ro_group;
GRANT SELECT ON ALL TABLES IN SCHEMA app TO ro_group;
" mydb

# Grant read-write access
run_sql "
GRANT USAGE, CREATE ON SCHEMA app TO rw_group;
GRANT SELECT, INSERT, UPDATE, DELETE
ON ALL TABLES IN SCHEMA app TO rw_group;
GRANT USAGE, SELECT ON ALL SEQUENCES
IN SCHEMA app TO rw_group;
" mydb
```

Default Privileges (for future objects)

```
# Ensure future tables created by app_owner are readable by ro_group
run_sql "
ALTER DEFAULT PRIVILEGES FOR ROLE app_owner
IN SCHEMA app
GRANT SELECT ON TABLES TO ro_group;

ALTER DEFAULT PRIVILEGES FOR ROLE app_owner
IN SCHEMA app
GRANT USAGE, SELECT ON SEQUENCES TO ro_group;
" mydb
```

Column-Level Grants

```
# Grant SELECT on specific columns only
run_sql "
REVOKE SELECT ON app.customers FROM analyst_role;
GRANT SELECT (customer_id, name, city)
ON app.customers TO analyst_role;
" mydb
```

Revoke Privileges

```
run_sql "
REVOKE ALL ON SCHEMA staging FROM temp_user;
REVOKE CONNECT ON DATABASE mydb FROM temp_user;
" mydb
```

3 User / Role Deletion

Removing a user from the CR's spec .users section does not drop the role from PostgreSQL. The operator explicitly leaves orphaned roles intact to prevent accidental data loss.

Safe Role Deletion Procedure

```
# Step 1: Remove user from CR (kubectl apply) – stops operator management

# Step 2: Reassign owned objects (in EVERY database the user has objects)
run_sql "REASSIGN OWNED BY old_user TO app_owner;" mydb

# Step 3: Drop remaining privileges
run_sql "DROP OWNED BY old_user;" mydb
# Repeat for each database where the user may have objects
run_sql "DROP OWNED BY old_user;" otherdb

# Step 4: Drop the role
run_sql "DROP ROLE IF EXISTS old_user;"
```

Caution: Always run REASSIGN OWNED before DROP OWNED to preserve table data. DROP OWNED without REASSIGN will drop all objects the user owns.

4 Database Deletion / Renaming

Same as users — removing a database from the CR does not drop it from PostgreSQL. Database renaming is also not a CR operation.

```
# Terminate all connections before dropping
run_sql "
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE datname = 'old_database'
AND pid != pg_backend_pid();
"

# Drop the database
run_sql "DROP DATABASE IF EXISTS old_database;"

# Rename a database (requires no active connections)
run_sql "ALTER DATABASE staging_db RENAME TO staging_v2;"
```

5 Extension Installation and Management

The operator's extensions loader can pull custom extension binaries from S3, but the actual CREATE EXTENSION / ALTER EXTENSION / DROP EXTENSION commands inside each database are manual.

```
# List available extensions
run_sql "SELECT name, default_version, installed_version
FROM pg_available_extensions
WHERE installed_version IS NOT NULL
ORDER BY name;" mydb

# Install common DBA extensions
run_sql "CREATE EXTENSION IF NOT EXISTS pg_stat_statements;" mydb
run_sql "CREATE EXTENSION IF NOT EXISTS pg_trgm;" mydb
run_sql "CREATE EXTENSION IF NOT EXISTS pgcrypto;" mydb
run_sql "CREATE EXTENSION IF NOT EXISTS pgaudit;" mydb

# Update an extension
run_sql "ALTER EXTENSION pg_stat_statements UPDATE;" mydb

# Drop an extension
run_sql "DROP EXTENSION IF EXISTS tablefunc;" mydb
```

6 Table, Index, and DDL Operations

The CR has zero awareness of anything below the database level. All CREATE TABLE, ALTER TABLE, CREATE INDEX, partitioning, materialized views, and every DDL statement is entirely outside the operator's scope.

```
# Create a partitioned table
run_sql "
CREATE TABLE app.events (
  id BIGSERIAL,
  event_date DATE NOT NULL,
  event_type TEXT,
  payload JSONB
) PARTITION BY RANGE (event_date);

CREATE TABLE app.events_2026_q1 PARTITION OF app.events
FOR VALUES FROM ('2026-01-01') TO ('2026-04-01');
CREATE TABLE app.events_2026_q2 PARTITION OF app.events
FOR VALUES FROM ('2026-04-01') TO ('2026-07-01');
" mydb

# Create indexes
run_sql "CREATE INDEX CONCURRENTLY idx_events_type
ON app.events (event_type);" mydb

# Create a materialized view
run_sql "
```

```
CREATE MATERIALIZED VIEW app.daily_summary AS
SELECT event_date, event_type, count(*)
FROM app.events
GROUP BY event_date, event_type;
" mydb
```

7

VACUUM, ANALYZE, and Maintenance Tasks

While autovacuum runs automatically inside PostgreSQL, targeted maintenance operations on specific tables — bloat remediation, forced ANALYZE after bulk loads, REINDEX for index corruption — require manual intervention.

```
# Targeted VACUUM ANALYZE on a bloated table
run_sql "VACUUM (VERBOSE, ANALYZE) app.large_table;" mydb

# VACUUM FULL (rewrites table – acquires ACCESS EXCLUSIVE lock)
run_sql "VACUUM FULL app.large_table;" mydb

# REINDEX a specific index
run_sql "REINDEX INDEX CONCURRENTLY app.idx_events_type;" mydb

# REINDEX entire database
run_sql "REINDEX DATABASE mydb;" mydb

# CLUSTER table on an index (sorts physical rows)
run_sql "CLUSTER app.events USING idx_events_date;" mydb

# Check table bloat
run_sql "
SELECT schemaname, relname,
n_dead_tup, n_live_tup,
round(n_dead_tup::numeric / NULLIF(n_live_tup, 0) * 100, 2)
AS dead_pct,
last_autovacuum, last_autoanalyze
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY n_dead_tup DESC
LIMIT 20;
" mydb
```

8

Query Performance Tuning

No CR support for pg_stat_statements management, EXPLAIN ANALYZE workflows, identifying and killing long-running queries, or setting per-user/per-database resource parameters.

```
# Top 10 queries by total execution time
run_sql "
SELECT LEFT(query, 80) AS query_preview,
calls,
```

```

round(total_exec_time::numeric, 2) AS total_ms,
round(mean_exec_time::numeric, 2) AS avg_ms,
rows
FROM pg_stat_statements
ORDER BY total_exec_time DESC
LIMIT 10;
" mydb

# Reset pg_stat_statements counters
run_sql "SELECT pg_stat_statements_reset();" mydb

# Find and terminate long-running queries (> 5 minutes)
run_sql "
SELECT pid, now() - query_start AS duration,
state, LEFT(query, 100) AS query
FROM pg_stat_activity
WHERE state != 'idle'
AND query_start < now() - interval '5 minutes'
ORDER BY duration DESC;
"

# Cancel a query gracefully
run_sql "SELECT pg_cancel_backend(<pid>);"

# Terminate a session forcefully
run_sql "SELECT pg_terminate_backend(<pid>);"

```

9

Logical Replication

The CR manages streaming (physical) replication via Patroni. Logical replication — publications, subscriptions, and logical replication slots — is entirely manual. This includes cross-cluster data sharing and selective table replication.

Ensure `wal_level` is set to 'logical' in the CR custom config first:

```

# In your PerconaPGCluster CR:
# spec:
# patroni:
# dynamicConfiguration:
# postgresql:
# parameters:
# wal_level: logical
# max_replication_slots: 10
# max_wal_senders: 10

```

Then manage publications and subscriptions via SQL:

```

# On the PUBLISHER (source cluster)
run_sql "
CREATE PUBLICATION my_pub FOR TABLE
app.customers, app.orders;

```

```
" mydb

# On the SUBSCRIBER (destination cluster)
run_sql "
CREATE SUBSCRIPTION my_sub
CONNECTION 'host=source-primary port=5432
dbname=mydb user=repl_user password=xxx'
PUBLICATION my_pub;
" mydb

# Monitor replication lag
run_sql "
SELECT subname, received_lsn, latest_end_lsn,
latest_end_time
FROM pg_stat_subscription;
"
```

10 Row-Level Security (RLS) Policies

RLS enables fine-grained access control at the row level. The CR has no concept of RLS — all policy creation, modification, and enablement is manual SQL.

```
# Enable RLS on a table
run_sql "
ALTER TABLE app.customer_data ENABLE ROW LEVEL SECURITY;
ALTER TABLE app.customer_data FORCE ROW LEVEL SECURITY;
" mydb

# Create a policy – users see only their own tenant's data
run_sql "
CREATE POLICY tenant_isolation ON app.customer_data
USING (tenant_id = current_setting('app.current_tenant')::int);
" mydb

# Policy for specific role
run_sql "
CREATE POLICY analyst_read ON app.customer_data
FOR SELECT TO analyst_role
USING (region = 'US');
" mydb
```

11 Tablespace Management (Logical)

The CR supports PVC volume definitions for tablespace storage, but the actual PostgreSQL CREATE TABLESPACE, moving tables/indexes between tablespaces, and logical tablespace management is manual.

```
# Create a tablespace (path must exist on the pod's filesystem)
run_sql "CREATE TABLESPACE fast_ssd"
```

```
LOCATION '/pgdata/tablespaces/fast_ssd';"

# Move a table to the new tablespace
run_sql "ALTER TABLE app.hot_data
SET TABLESPACE fast_ssd;" mydb

# Move an index
run_sql "ALTER INDEX app.idx_hot_data_pk
SET TABLESPACE fast_ssd;" mydb

# Set default tablespace for a database
run_sql "ALTER DATABASE mydb SET default_tablespace = fast_ssd;"
```

12 Foreign Data Wrappers (FDW)

Connecting to external data sources (other PostgreSQL instances, MySQL, flat files) via FDW is entirely manual. The operator has no awareness of foreign servers or tables.

```
# Install the FDW extension
run_sql "CREATE EXTENSION IF NOT EXISTS postgres_fdw;" mydb

# Create a foreign server
run_sql "
CREATE SERVER legacy_server
FOREIGN DATA WRAPPER postgres_fdw
OPTIONS (host 'legacy-db.internal',
port '5432', dbname 'legacy_app');
" mydb

# Create user mapping
run_sql "
CREATE USER MAPPING FOR app_user
SERVER legacy_server
OPTIONS (user 'fdw_reader', password 'xxx');
" mydb

# Import foreign schema
run_sql "
IMPORT FOREIGN SCHEMA public
LIMIT TO (customers, orders)
FROM SERVER legacy_server INTO app_foreign;
" mydb
```

13 Event Triggers and Regular Triggers

All trigger functions, row-level triggers, statement-level triggers, and event triggers (DDL audit hooks) must be created via SQL.

```
# Create an audit trigger function
run_sql "
CREATE OR REPLACE FUNCTION audit.log_changes()
RETURNS TRIGGER AS \$$
BEGIN
INSERT INTO audit.change_log
(table_name, operation, old_data, new_data, changed_by, changed_at)
VALUES (
TG_TABLE_NAME, TG_OP,
row_to_json(OLD), row_to_json(NEW),
current_user, now()
);
RETURN NEW;
END;
\$$ LANGUAGE plpgsql;
" mydb

# Attach the trigger
run_sql "
CREATE TRIGGER trg_customers_audit
AFTER INSERT OR UPDATE OR DELETE ON app.customers
FOR EACH ROW EXECUTE FUNCTION audit.log_changes();
" mydb

# Event trigger for DDL auditing
run_sql "
CREATE OR REPLACE FUNCTION audit.log_ddl()
RETURNS event_trigger AS \$$
BEGIN
INSERT INTO audit.ddl_log (event, tag, username, ts)
VALUES (tg_event, tg_tag, current_user, now());
END;
\$$ LANGUAGE plpgsql;

CREATE EVENT TRIGGER trg_ddl_audit
ON ddl_command_end
EXECUTE FUNCTION audit.log_ddl();
" mydb
```

14

Scheduled Jobs Inside PostgreSQL (pg_cron)

The CR's scheduling capability is limited to pgBackRest backup crons. For in-database scheduling (partition maintenance, stats refresh, cleanup jobs), you need pg_cron.

First, add pg_cron to shared_preload_libraries via the CR:

```
# In your PerconaPGCluster CR:
# spec:
# patroni:
# dynamicConfiguration:
# postgresql:
```

```
# parameters:
# shared_preload_libraries:
# pg_stat_statements,pg_cron
# cron.database_name: postgres
```

Then create and manage jobs via SQL:

```
# Install the extension
run_sql "CREATE EXTENSION IF NOT EXISTS pg_cron;"

# Schedule nightly VACUUM ANALYZE
run_sql "
SELECT cron.schedule(
'nightly-vacuum',
'0 3 * * *',
\$$VACUUM ANALYZE app.large_table\$$
);
"

# Schedule partition creation (run monthly)
run_sql "
SELECT cron.schedule(
'create-next-partition',
'0 0 25 * *',
\$$SELECT create_next_month_partition()\$$
);
"

# List scheduled jobs
run_sql "SELECT jobid, schedule, command, nodename
FROM cron.job;"

# Remove a job
run_sql "SELECT cron.unschedule('nightly-vacuum');"
```

15

Audit Configuration (pgAudit)

While pgaudit parameters can be set via the CR's custom config, the extension installation and per-role/per-object audit settings are manual.

```
# Install pgAudit
run_sql "CREATE EXTENSION IF NOT EXISTS pgaudit;" mydb

# Set global audit logging (via CR or ALTER SYSTEM)
# In CR: pgaudit.log = 'ddl, role'

# Per-role audit settings
run_sql "ALTER ROLE app_admin SET pgaudit.log = 'all';"
run_sql "ALTER ROLE app_user SET pgaudit.log = 'write, ddl';"
```

```
# Object-level auditing
run_sql "
-- Create an audit role
CREATE ROLE pg_auditor NOLOGIN;
-- Grant access to audited objects
GRANT SELECT ON app.sensitive_data TO pg_auditor;
-- Set the object audit role
ALTER SYSTEM SET pgaudit.role = 'pg_auditor';
SELECT pg_reload_conf();
"
```

16 Connection-Level Parameter Overrides

Per-role or per-database `ALTER ROLE SET` / `ALTER DATABASE SET` for parameters like `work_mem`, `statement_timeout`, `lock_timeout` are not supported through the CR.

```
# Per-role resource limits
run_sql "ALTER ROLE etl_user SET work_mem = '512MB';"
run_sql "ALTER ROLE etl_user SET maintenance_work_mem = '1GB';"
run_sql "ALTER ROLE etl_user SET statement_timeout = '3600000';"

# Per-role safety limits for app users
run_sql "ALTER ROLE app_user SET statement_timeout = '30000';"
run_sql "ALTER ROLE app_user SET lock_timeout = '5000';"
run_sql "ALTER ROLE app_user SET idle_in_transaction_session_timeout = '60000';"

# Per-database defaults
run_sql "ALTER DATABASE analytics SET work_mem = '256MB';"
run_sql "ALTER DATABASE analytics SET effective_cache_size = '8GB';"
```

17 Large Object Management

PostgreSQL large object operations (`lo_import`, `lo_export`) and cleanup with `vacuumlo` are entirely outside the operator's scope.

```
# Import a large object
kubectrl exec -n "$NAMESPACE" "$PRIMARY" -c database -- \
psql -U postgres -d mydb -c "
SELECT lo_import('/tmp/large_file.bin');
"

# Export a large object
kubectrl exec -n "$NAMESPACE" "$PRIMARY" -c database -- \
psql -U postgres -d mydb -c "
SELECT lo_export(12345, '/tmp/exported_file.bin');
"

# Clean up orphaned large objects
```

```
kubectl exec -n "$NAMESPACE" "$PRIMARY" -c database -- \
vacuumlo -U postgres mydb
```

18 Database Cloning / Template Databases

The CR creates databases from the default template (template1) only. Custom template databases and cloning are manual.

```
# Create a template database with standard schema
run_sql "CREATE DATABASE app_template OWNER postgres;"
run_sql "CREATE SCHEMA IF NOT EXISTS app;" app_template
run_sql "CREATE SCHEMA IF NOT EXISTS audit;" app_template
run_sql "
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
CREATE EXTENSION IF NOT EXISTS pgcrypto;
" app_template

# Mark it as a template
run_sql "
UPDATE pg_database SET datistemplate = true
WHERE datname = 'app_template';
"

# Clone from template
run_sql "CREATE DATABASE new_app_db
TEMPLATE app_template
OWNER app_user;"
```

19 Statistics and Planner Hints

ALTER TABLE SET STATISTICS, CREATE STATISTICS (extended multi-column statistics), and planner hint management are not supported through the CR.

```
# Increase statistics target for a skewed column
run_sql "
ALTER TABLE app.orders
ALTER COLUMN status SET STATISTICS 1000;
ANALYZE app.orders;
" mydb

# Create extended statistics for correlated columns
run_sql "
CREATE STATISTICS app.stats_orders_region_status
(dependencies, ndistinct, mcv)
ON region, status FROM app.orders;
ANALYZE app.orders;
" mydb
```

```
# View extended statistics
run_sql "
SELECT stxname, stxkeys, stxkind
FROM pg_statistic_ext
WHERE stxnamespace = 'app'::regnamespace;
" mydb
```

20 Patroni Direct Interventions

While the CR exposes some Patroni settings (e.g., enable/disable switchover), direct Patroni operations like manual switchover, failover, reinit, pause/resume HA, and member list inspection require exec-ing into the pod and running `patronictl`.

```
# List cluster members
kubectl exec -n "$NAMESPACE" "$$PRIMARY" -c database -- \
patronictl list

# Planned switchover (promote a specific replica)
kubectl exec -n "$NAMESPACE" "$$PRIMARY" -c database -- \
patronictl switchover "$$CLUSTER" \
--leader "$$PRIMARY" \
--candidate my-cluster-instance-xxxx-0 \
--force

# Emergency failover (when primary is unreachable)
kubectl exec -n "$NAMESPACE" <any-replica-pod> -c database -- \
patronictl failover "$$CLUSTER" --force

# Reinitialize a broken replica
kubectl exec -n "$NAMESPACE" "$$PRIMARY" -c database -- \
patronictl reinit "$$CLUSTER" <broken-replica-name> --force

# Pause HA (maintenance mode — prevents automatic failover)
kubectl exec -n "$NAMESPACE" "$$PRIMARY" -c database -- \
patronictl pause "$$CLUSTER" --wait

# Resume HA
kubectl exec -n "$NAMESPACE" "$$PRIMARY" -c database -- \
patronictl resume "$$CLUSTER" --wait

# Show Patroni configuration
kubectl exec -n "$NAMESPACE" "$$PRIMARY" -c database -- \
patronictl show-config "$$CLUSTER"

# Reload Patroni configuration
kubectl exec -n "$NAMESPACE" "$$PRIMARY" -c database -- \
patronictl reload "$$CLUSTER"
```

Caution: Always pause Patroni HA before performing maintenance that could trigger a false failover (e.g., **VACUUM FULL** on large tables, parameter changes requiring restart). Resume HA immediately after maintenance

completes.

PART IV — AUTOMATION PATTERNS

Three Approaches for Automating Manual DBA Tasks

Approach 1: Kubernetes CronJob (Recommended)

Deploy a CronJob that runs your DBA scripts on a schedule inside the cluster. This is the most self-contained approach with no external dependencies.

```
apiVersion: batch/v1
kind: CronJob
metadata:
  name: dba-daily-tasks
  namespace: pg-production
spec:
  schedule: "30 6 * * *"
  successfulJobsHistoryLimit: 7
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      template:
        spec:
          serviceAccountName: dba-automation
          containers:
            - name: dba-runner
              image: bitnami/kubectl:latest
              command: ["/bin/bash", "/scripts/dba-tasks.sh"]
              volumeMounts:
                - name: scripts
                  mountPath: /scripts
          volumes:
            - name: scripts
              configMap:
                name: dba-scripts
                defaultMode: 0755
              restartPolicy: OnFailure
```

Required RBAC for the CronJob ServiceAccount:

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: dba-automation
  namespace: pg-production
---
apiVersion: rbac.authorization.k8s.io/v1
```

```
kind: Role
metadata:
  name: dba-automation-role
  namespace: pg-production
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list"]
- apiGroups: [""]
  resources: ["pods/exec"]
  verbs: ["create"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: dba-automation-binding
  namespace: pg-production
subjects:
- kind: ServiceAccount
  name: dba-automation
roleRef:
  kind: Role
  name: dba-automation-role
apiGroup: rbac.authorization.k8s.io
```

Approach 2: External Cron on Bastion/Admin VM

If you have a management VM with kubectl configured, schedule DBA scripts via system cron. This is closest to traditional VM-based DBA workflows and simplest to set up.

```
# /etc/cron.d/pg-dba-tasks
30 6 * * * dba_user /opt/dba/scripts/daily-maintenance.sh \
>> /var/log/dba/daily-maintenance.log 2>&1
0 2 * * 0 dba_user /opt/dba/scripts/weekly-vacuum-full.sh \
>> /var/log/dba/weekly-vacuum.log 2>&1
```

Approach 3: CI/CD Pipeline (GitOps)

Store SQL scripts in a Git repository. Trigger DBA tasks via merge to a designated branch. Best for auditable, change-tracked operations.

```
# .gitlab-ci.yml example
dba-deploy:
  stage: deploy
  image: bitnami/kubectl:latest
  script:
  - kubectl config use-context production
  - bash scripts/apply-schema-changes.sh
  only:
```

```
- main  
when: manual
```

PART V — COMMON kubectl exec WRAPPER SCRIPT

Use this reusable wrapper script as the foundation for all DBA automation. Store it in a ConfigMap or on your bastion VM.

```
#!/bin/bash
# pg-dba-wrapper.sh — Reusable DBA wrapper for Percona Operator clusters
set -euo pipefail

NAMESPACE="${PG_NAMESPACE:-pg-production}"
CLUSTER="${PG_CLUSTER:-my-cluster}"
CONTAINER="database"
LOG_DIR="${LOG_DIR:-/tmp/dba-logs}"
mkdir -p "$LOG_DIR"

# Identify primary pod
get_primary() {
  kubectl get pod -n "$NAMESPACE" \
  -l postgres-operator.crunchydata.com/role=master, \
  postgres-operator.crunchydata.com/cluster="$CLUSTER" \
  -o jsonpath='{.items[0].metadata.name}'
}

PRIMARY=$(get_primary)
if [[ -z "$PRIMARY" ]]; then
  echo "ERROR: Cannot find primary pod" | tee -a "$LOG_DIR/errors.log"
  exit 1
fi

echo "[$(date)] Primary pod: $PRIMARY" | tee -a "$LOG_DIR/dba.log"

# Execute SQL on primary
run_sql() {
  local sql="$1"
  local db="${2:-postgres}"
  echo "[$(date)] Running on $db: ${sql:0:80}..." \
  | tee -a "$LOG_DIR/dba.log"
  kubectl exec -n "$NAMESPACE" "$PRIMARY" -c "$CONTAINER" -- \
  psql -U postgres -d "$db" -c "$sql" 2>&1 \
  | tee -a "$LOG_DIR/dba.log"
}

# Execute SQL file on primary
run_sql_file() {
  local file="$1"
  local db="${2:-postgres}"
  echo "[$(date)] Running file $file on $db" \
  | tee -a "$LOG_DIR/dba.log"
  kubectl exec -i -n "$NAMESPACE" "$PRIMARY" -c "$CONTAINER" -- \
  psql -U postgres -d "$db" < "$file" 2>&1 \
  | tee -a "$LOG_DIR/dba.log"
}
```

```
# Run patronictl command
run_patronictl() {
  kubectl exec -n "$NAMESPACE" "$PRIMARY" -c "$CONTAINER" -- \
  patronictl "$@"
}

echo "[$(date)] DBA wrapper initialized for $CLUSTER in $NAMESPACE"
```

PART VI — QUICK REFERENCE CHEAT SHEET

Task	CR?	Manual Command
Create user	Yes	spec.users in CR
Delete user	No	DROP OWNED + DROP ROLE
Create database	Yes	spec.users.databases in CR
Drop database	No	DROP DATABASE
Create schema	No	CREATE SCHEMA
Grant privileges	No	GRANT ... ON ... TO ...
Install extension	No	CREATE EXTENSION
Schedule backup	Yes	spec.backups.schedules
Ad-hoc backup	Yes	PerconaPGBBackup CR
VACUUM FULL	No	VACUUM FULL table
Kill query	No	pg_terminate_backend(pid)
Logical replication	No	CREATE PUBLICATION/SUBSCRIPTION
pg_cron job	No	cron.schedule()
RLS policy	No	CREATE POLICY
Switchover	Partial	patronictl switchover
Scale replicas	Yes	spec.instances.replicas
TLS certs	Yes	cert-manager auto
pgBouncer config	Yes	spec.proxy.pgBouncer
ALTER ROLE SET	No	ALTER ROLE x SET param=val

Key Takeaway: The Percona Operator handles the infrastructure plane (provisioning, HA, backups, scaling, TLS). The entire data plane (schemas, privileges, DDL, maintenance, tuning, logical replication, auditing, scheduling) remains traditional DBA work requiring kubectl exec access to the pods. Automate these tasks with Kubernetes CronJobs, bastion scripts, or CI/CD pipelines using the wrapper script in Part V as your foundation.